CAMPUS SMS

System Documentation

School Management System: database schema, architecture and flowcharts

Computer Science · Computer Engineering · Entrepreneurship Version 1.0 · October 10, 2026

Contents

1	Overview, roles and technology
2	System architecture (visual flow)
3	Database schema
4	Flowcharts: accounts, enrollment, appointment tickets
5	Roles, permissions and sidebar menus
6	API overview
7	Running the project and sample accounts

1. Overview

Campus SMS is a small school management system. Students register and wait for an Admin to approve the account. After that they request classes, see their grades, check their tuition balance and book appointment tickets for the registrar or accounting office. The Admin oversees everything and can approve, deny, edit or delete almost any record. The school offers three programs: **Computer Science**, **Computer Engineering** and **Entrepreneurship**.

Role	What they do
Admin	Approves or denies accounts, enrollments and tickets. Creates, edits, deactivates and deletes users, courses, sections, enrollments and tickets. Edits tuition ledgers and grades.
Registrar	Manages courses, class sections and enrollment requests. Handles registration and inquiry tickets.
Accounting	Verifies reference numbers at the window, records cash received and updates tuition ledgers.
Faculty	Sees their own classes and rosters and enters grades.
Student	Browses classes by program, requests and drops classes, views grades and balance, books appointments.

Technology

Layer	Technology	Why
Frontend	React + Vite, Tailwind CSS, React Router	Fast to build, responsive, sidebar layout with a dashboard per role
Backend	Node.js + Express (one API)	Simple: one server instead of three separate ones
Database	PostgreSQL 16 in Docker (+ pgAdmin)	Reliable relational data, easy to run anywhere
Security	bcrypt hashes, JWT tokens (8 h), role checks on every route	Standard and easy to understand

2. System architecture

The visual flow below shows how a request travels. Everyone uses the React website. The website talks to one Express API, which checks the login token and the role, then reads or writes the PostgreSQL database running in Docker.

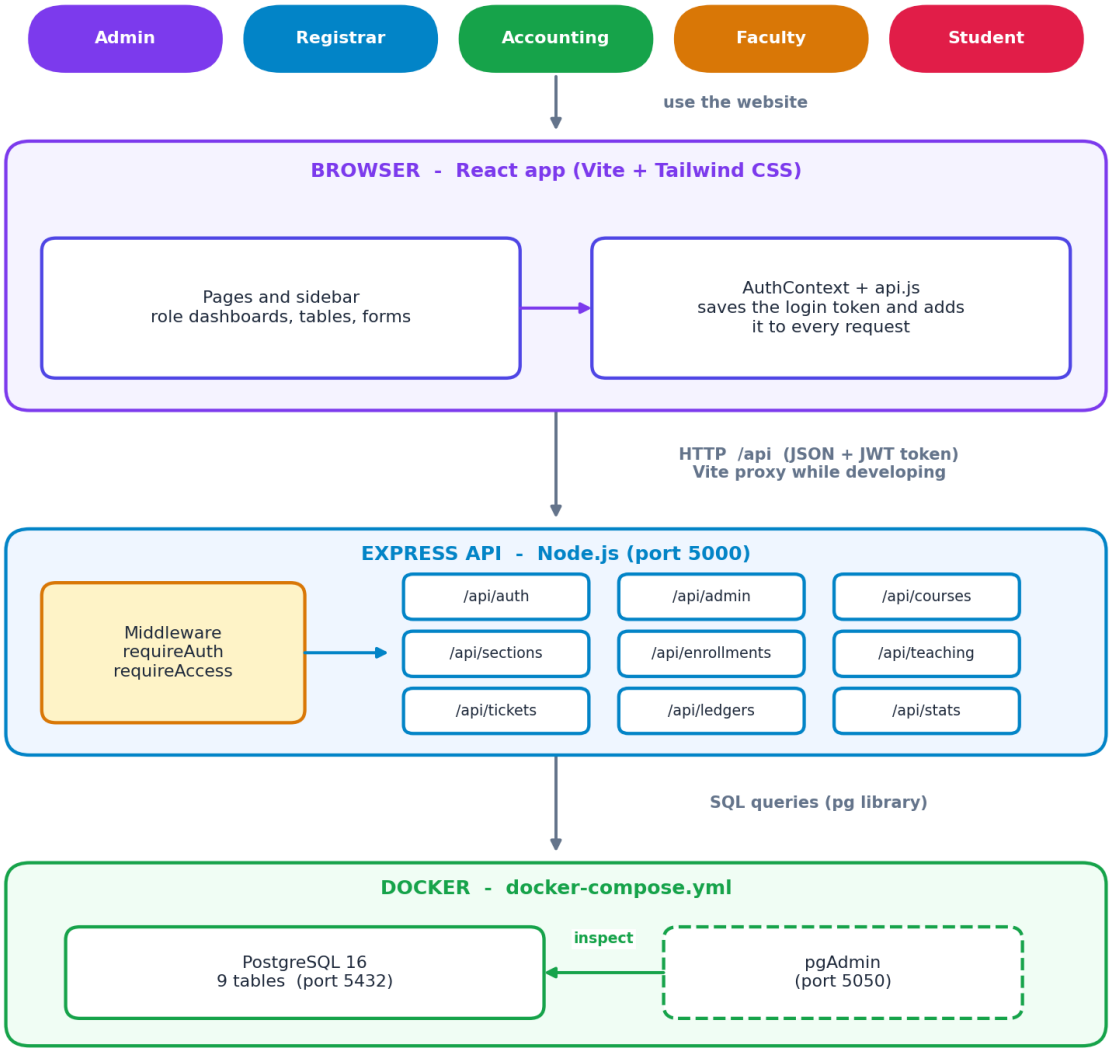


Figure 1. System architecture

How one request works

- The person signs in. The API checks the password and returns a **JWT token** valid for 8 hours.
- **api.js** stores the token and adds it to every request as a header.
- The **requireAuth** middleware checks that the token is valid. **requireAccess** checks that the role may use that route.
- The route runs a SQL query through the **pg** library and sends the result back as JSON.
- During development the Vite server forwards **/api** to port 5000, so the browser only talks to one address.

3. Database schema

Nine tables. Every person is a row in **users**. A profile table adds the details for their role. The **status** columns (pending, approved, denied) power the Admin approve and deny buttons.

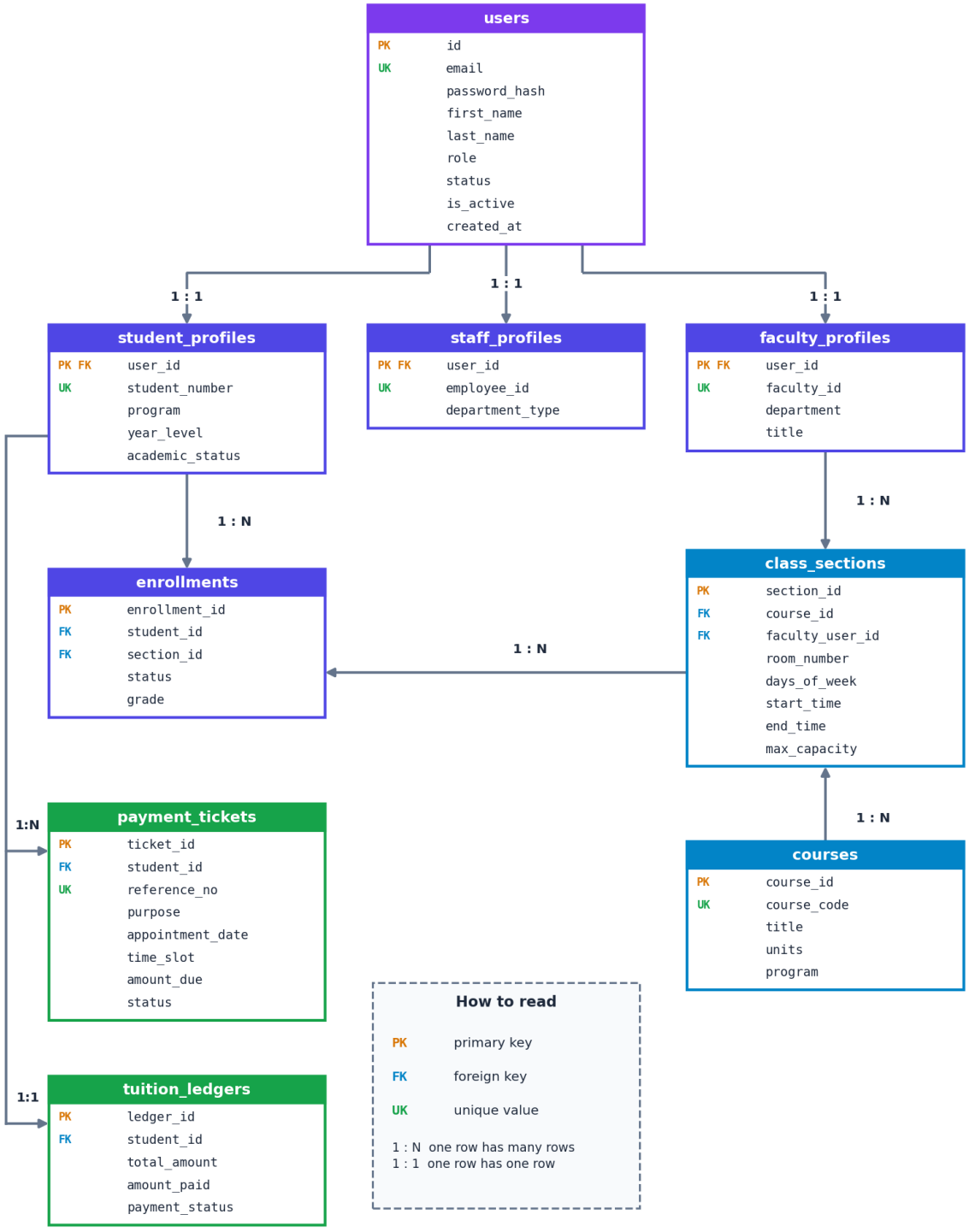


Figure 2. Entity relationship diagram

Table guide

Table	Purpose	Key rules
users	Login accounts for everyone.	role: admin, staff, faculty, student. status: pending, approved, denied. is_active lets the Admin deactivate.
student_profiles	Student number, program, year level.	program must be Computer Science, Computer Engineering or Entrepreneurship.
staff_profiles	Employee ID and department.	department_type: registrar or accounting.
faculty_profiles	Faculty ID, department, title.	Deleting a faculty member leaves their sections without a teacher (SET NULL).
courses	The course catalog.	Every course belongs to one of the three programs.
class_sections	A course offered with a teacher, room, schedule and seat limit.	Sections can be filtered by program on the Enroll page.
enrollments	A student request to join a section. Holds the grade.	status: pending, approved, denied, dropped. One row per student and section.
payment_tickets	Appointment booking with a unique reference number.	purpose: accounting, registration, inquiry. status: pending, ready, completed, denied.
tuition_ledgers	One running balance per student.	payment_status: unpaid, partial, paid. Created when the student is approved.

Delete behaviour

Most links use **ON DELETE CASCADE**. Deleting a user also deletes their profile, enrollments, tickets and ledger. Deleting a course deletes its sections and their enrollments. This keeps the data clean, so the Admin sees a confirmation before deleting.

4. Flowcharts

Three flows cover the main processes. Each one shows who acts, which decisions are made and which status the record ends with.

4.1 Account approval and login

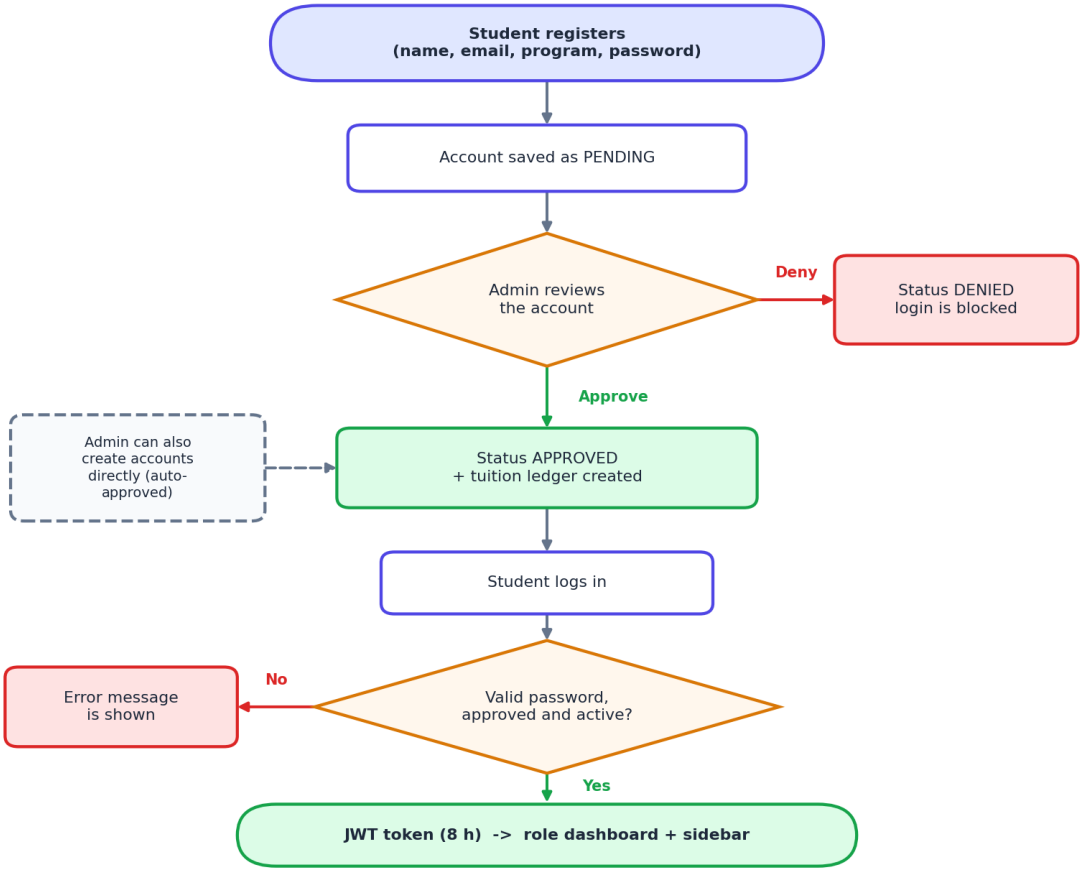


Figure 3. Account approval and login

- A student registers. The account is saved as **pending** and cannot log in yet.
- The Admin approves or denies it. Approving also creates the student tuition ledger.
- Login fails with a clear message if the account is pending, denied, deactivated or the password is wrong.
- On success the API returns a JWT token and the app opens the dashboard and sidebar for that role.

4.2 Enrollment and grading

A student browses the classes (filterable by program) and requests a section. The Registrar or Admin decides, and the seat limit is checked on approval.

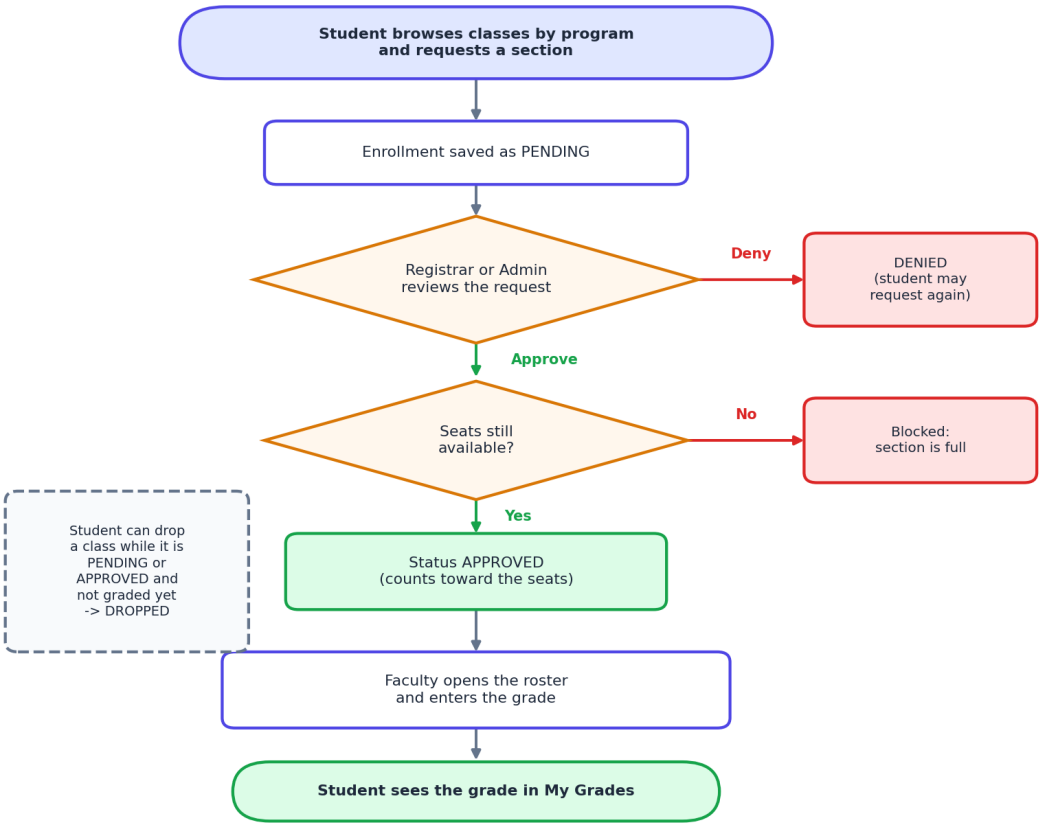


Figure 4. Enrollment and grading

Step by step

- The student sends a request. The enrollment is saved as **pending**.
- The Registrar or Admin approves or denies it. A denied student may request the class again.
- Approval is blocked when the section is already full.
- Faculty open the roster of their own sections and enter grades. Admin can edit any grade.
- The student sees the grade under My Grades. A student can drop a class that is pending or approved and not graded yet.

4.3 Appointment tickets

This follows the original design: the student books remotely, sees the status on the ticket, then finishes the transaction at the window with the reference number.

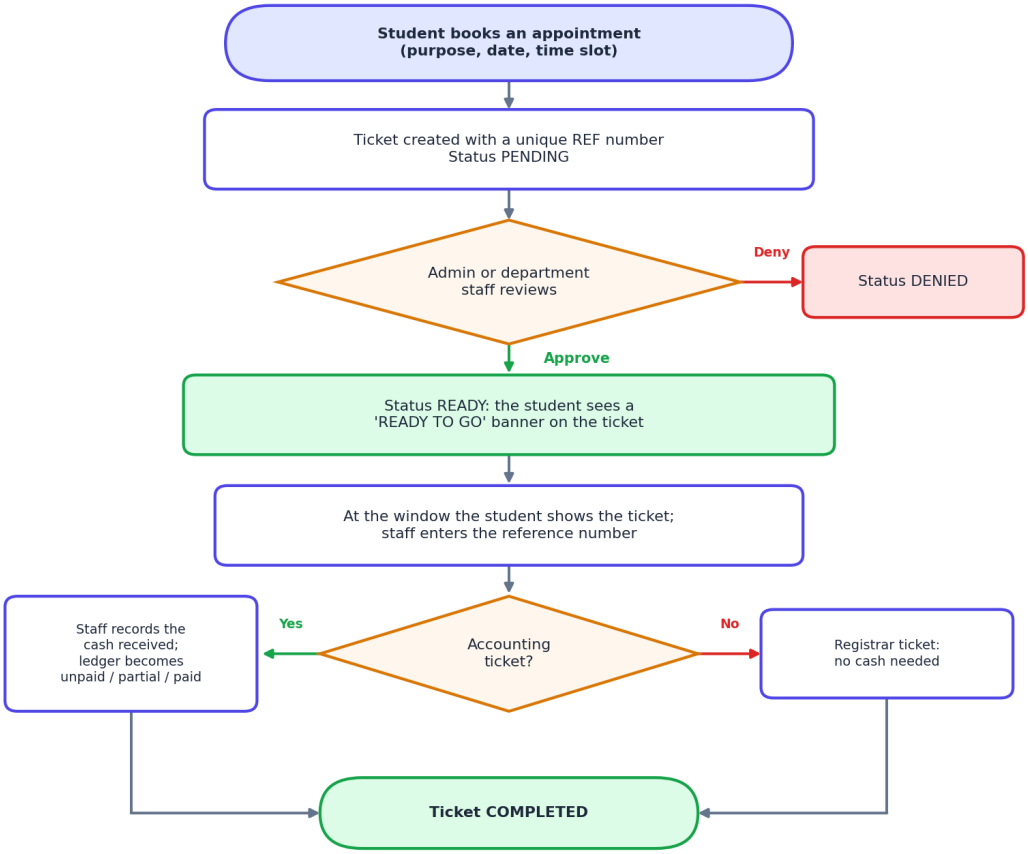


Figure 5. Appointment ticket flow

Step by step

- The student chooses a purpose (accounting, registration or inquiry), a date and a time slot. The system creates a unique reference number such as REF-2026-12345.
- Admin or the right department approves the ticket (status **ready**) or denies it. Registrar handles registration and inquiry tickets, Accounting handles accounting tickets.
- At the window the staff member types the reference number to find the ticket.
- For an accounting ticket the staff member enters the cash received. The tuition ledger is updated to unpaid, partial or paid.
- The ticket becomes **completed**. Admin can edit or delete any ticket.

5. Roles, permissions and sidebar menus

Feature	Admin	Registrar	Accounting	Faculty	Student
Approve, deny, create, edit, delete users	✓				
Courses and sections	✓	✓			
Approve, deny, delete enrollments	✓	✓			
Request and drop classes					✓
Rosters and grades	✓ all			✓ own	
View own grades					✓
Book appointments, view own tickets					✓
Approve, deny tickets	✓	✓ reg.	✓ acct.		
Verify reference number, complete ticket	✓	✓ reg.	✓ acct.		
Edit or delete tickets	✓				
View and edit tuition ledgers	✓		✓		
View own balance					✓

Registrar and Accounting both use the staff role. Their department decides which tickets they can handle (reg. = registration and inquiry, acct. = accounting).

Sidebar menu for each role

Role	Sidebar items
Admin	Dashboard · Users · Courses & Sections · Enrollments · Classes & Grades · Tickets · Tuition Ledgers
Registrar	Dashboard · Courses & Sections · Enrollments · Tickets
Accounting	Dashboard · Tickets · Tuition Ledgers
Faculty	Dashboard · My Classes
Student	Dashboard · Enroll in Classes · My Grades · Appointments

Each role also gets its own dashboard with live numbers, such as pending approvals for the Admin, grades still to enter for Faculty and the tuition balance for Students.

6. API overview

Route	Purpose	Who can use it
/api/auth	Register, login, who am I	Everyone
/api/admin/users	List, create, edit, approve or deny, delete users	Admin
/api/courses	Course catalog	Read: all. Write: Registrar, Admin
/api/sections	Class sections (with their program)	Read: all. Write: Registrar, Admin
/api/enrollments	Request, drop, approve, deny, delete	Student (own), Registrar, Admin
/api/teaching	Own sections, rosters, grades	Faculty (own), Admin
/api/tickets	Book, approve, verify, complete, edit, delete	Student (own), staff by department, Admin
/api/ledgers	Tuition balances	Student (own), Accounting, Admin
/api/stats	Numbers for the dashboards	By role

7. Running the project

Folders

```
School-management-system/
├── docker-compose.yml        PostgreSQL + pgAdmin
├── database/schema.sql       creates the 9 tables
├── server/                   Express API
│   ├── index.js  utils.js  db.js  seed.js  .env
│   ├── middleware/authMiddleware.js
│   └── routes/               auth, admin, courses, sections, enrollments,
│                             teaching, tickets, ledgers, stats
└── client/src/               React app
    ├── App.jsx  main.jsx  api.js  hooks.js  AuthContext.jsx
    ├── components/           Shell (sidebar), ui, charts, Modal, Feedback ...
    └── pages/                Dashboard, Login, Register and one folder per role
```

Start-up commands

```
# 1. database (Docker Desktop must be running)
docker compose up -d
Get-Content database\schema.sql | docker compose exec -T db psql -U sms_user -d sms_db

# 2. sample data (WARNING: schema.sql wipes existing data)
cd server
node seed.js

# 3. API
npm run dev            # http://localhost:5000

# 4. website (new terminal)
cd client
npm install lucide-react    # only needed once
npm run dev            # open the address it prints
```

Sample data

The seed script creates 6 faculty, **9 students** (8 approved and 1 waiting for approval), 18 courses (6 per program), class sections, enrollments with grades, tuition ledgers and appointment tickets in different statuses. Every account uses the password **password123** (development only).

Email	Role	Notes
admin@sms.com	Admin	Approve accounts, full control
registrar@sms.com	Staff, registrar	Courses, sections, enrollments
accounting@sms.com	Staff, accounting	Tickets and tuition ledgers
faculty@sms.com	Faculty	Computer Science instructor
student@sms.com	Student	Juan Dela Cruz, Computer Science
newstudent@sms.com	Student	Maria Santos, pending: approve her as Admin

Before real use

- Change **JWT_SECRET** in server/.env to a long random value.
- Delete the sample accounts and use strong passwords.
- Run **npm audit** in both folders and update packages.